

Grafts, Depth, and Diagnostics

A 28M-parameter encoder under a hard parameter cap:
what two weeks of laptop-scale experiments established

Osman Siddique

Claude

August 2026

benchtop-bert

Abstract

We pretrained a 28M-parameter BERT-style encoder from scratch—attention, blocks, embeddings and MLM head, only the tokenizer borrowed—on a single 4 GB laptop GPU under a hard 30M-parameter cap. **Five results transfer beyond this scale.**

1. **Zero-init layer grafting is exact in post-LN transformers**, not only in pre-LN as the folklore holds—provided the block is appended at the *top* of the stack and its LayerNorm affines are *reset* to $(\gamma=1, \beta=0)$ rather than copied from the donor. Copying them, which is what progressive stacking does, costs **+4.12 nats**. Resetting them costs **0.000 nats** ($\max |\Delta \text{logit}| = 1.3 \times 10^{-3}$ against a mean logit of 3.82). Mid-stack insertion costs **+0.23 nats** however it is initialised. (§4)
2. **A training run can be fully instrumented at $\approx$ zero marginal GPU cost**. Atomic checkpoints make a live run readable from outside; the diagnostics then score on the **idle CPU in 8.9–22.2 s** each, against **13.2–16.6 minutes of exclusive GPU** for a downstream evaluation. ≈ 100 readings per metric over 35 GPU-hours is what made trends legible where single readings were actively misleading. (§6.1)
3. **Effective rank designed the architecture, and the prediction held**. FFN activations occupied 133–866 of 3,072 directions while residual-stream rank was *still climbing* at the last layer; width was cut $3072 \rightarrow 1792$ and spent on depth; re-measuring the trained result gave $r_{99}/d_{\text{ff}} \leq 0.56$, confirming width was still not the binding constraint. (§6.2)
4. **Replay floor beats replay fraction**. 10% replay retained as well as 35%, while the one corpus dropped to 0% lost **0.95 nats**—four times as much. (§3)
5. **MLM loss and downstream accuracy decoupled completely at this scale**. A -0.435 nats held-out improvement ($\approx 9\times$ eval noise, unambiguously real) moved SST-2 by **0.0 pp**. The spread across three very different pretraining regimes is **1.1 SE**, and the two SST-2 harnesses disagree with each other by more than any intervention moved the number. (§11)

Also reported: a sixth layer with no work to do under three independent initialisations; a *negative* pairwise ablation marginal—removing a useless layer halves the measured cost of removing a useful one; an induction circuit installed into a masked LM, taking copy-from-context from 0.8% to 93% in 6.5 minutes; span masking that improved the harder task more than the easier one; and an $11\times$ effect of annealing *corpus* over annealing *schedule*. Total cost: **35.4 GPU-hours**, $\approx 3.0\text{B}$ tokens processed.

Contents

1 Introduction

3

2	Setup	4
2.1	Model	4
2.2	Data	4
2.3	Training	4
2.4	Evaluation protocol	4
3	What worked	5
3.1	Data curriculum: the largest single lever in the project	5
3.2	Replay mixing works, and we have the control	5
3.3	Span masking	6
3.4	WSD with the decay phase left unspent	6
3.5	Installing an induction circuit into a masked language model	6
3.6	The diagnostic tooling	7
4	Function-preserving grafts in post-LN transformers	7
4.1	The problem	7
4.2	Measurement	7
4.3	Why it works, and where it stops working	8
4.4	The counter-intuitive term	8
4.5	Practical rules	8
5	Depth at 28M: the sixth layer	9
5.1	Ablation cost: a causal metric	9
5.2	Three initialisations, one answer	9
5.3	Pairwise marginals change how ablation should be read	9
6	Diagnostics: zero-cost telemetry, and what it can and cannot tell you	10
6.1	Zero-cost live diagnostics	10
6.2	What the cheap metrics got right	11
6.3	An unmeasured null: head similarity read ≈ 0.97 at random init	12
6.4	The same failure one level up: latent-target pretraining	12
6.5	A degenerate case: zero-output layers poison run-level aggregates	12
6.6	The general lesson	13
7	Annealing: corpus dominates schedule	13
8	Evaluation validity	13
9	Width–depth reallocation	14
10	Engineering results	15
11	Scoreboard, and the central negative result	16
11.1	Pretraining progressed clearly	16
11.2	Downstream did not	16
12	What we would do differently	17
13	Limitations	17
A	Reproducing the headline measurements	17

1 Introduction

The constraint that shaped this project was a hard cap of 30M parameters, on a single RTX 3050 Ti Laptop GPU with 4GB of VRAM. Under such a cap the interesting question is not *how well can this do*—the answer is known to be “worse than BERT-base”—but *how much of the gap can be closed by spending the budget better*.

The answer turned out to be “almost none of it” (§11), and the durable output is therefore not the model but **the instruments built to interrogate it**. Every result in this report comes from one of these, and each is a standalone script.

tool	what it measures	what it established
<code>graft_ladder.py</code>	whether a layer graft preserves the function, at the logit level	§4 — reset the donor’s LN affines: +4.12 $\rightarrow$ 0.000 nats
<code>layer_contrib.py</code>	causal ablation cost per layer, single and pairwise	§5 — layer 4 worth 0.073 vs layer 5’s 0.963; $\text{marginal}(4 3) = -1.891$
<code>ffn_rank.py</code>	activation and weight spectra, effective rank, $r@90/99\%$	§6.2 — d_ff 3.5 $\times$ oversized $\Rightarrow$ the entire v2 architecture
<code>dead_neuron_check.py</code>	per-unit firing rates over 2.0M tokens	2,697/3,072 layer-0 units dead $\Rightarrow$ GELU swap, and prune scored under GELU
<code>attn_health.py</code>	per-head entropy / contribution, out_rank, centred head similarity	§6 — the raw metric’s null is 0.97, not 0
<code>ffn_slack.py</code>	slack, never-positive fraction, utilisation, kurtosis	live per-layer health; separates dead-zone mass from real width pressure
<code>span_eval.py</code>	fixed-protocol held-out MLM loss, both masking schemes	the only cross-run comparable numbers in the repo (§2.4)
<code>audit_eval.py</code>	train/val overlap, duplicate blocks, averaging bias	§8 — found a genuinely contaminated corpus
<code>masking_difficulty.py</code>	scattered vs span difficulty offsets on one model	+1.74 nats — the train/eval gap that looks like a bug and is not

Table 1: The toolkit. All but the first two run on the **CPU in under 25 seconds** (§6.1), so they can be run against a live training job on a loop without costing it anything.

This report is organised around what the experiments established rather than around the chronology. Section 2 gives the setup and the evaluation protocol, which turned out to matter more than most of the interventions. Section 3 reports the components that demonstrably worked, with controls. Section 4 is the main technical contribution. Section 5 reports the depth result, and Section 6 the diagnostics. Sections 7–9 report the remaining results, Section 10 the engineering findings, and Section 11 the scoreboard and the central negative result.

Full checkpoint lineage and the chronological build log are in the repository, in `checkpoints/HISTORY.md` and `WORKLOG.md` respectively.

2 Setup

2.1 Model

	v1 line	v2 line (final)
parameters	28.733M	27.952M
layers / heads	4 / 4	6 / 4
<code>d_model</code> , $d_k = d_v$	768, 64	768, 64
<code>d_ff</code> , activation	3072, ReLU	1792, GELU
embedding	ALBERT-factorised $V \times 128$ then $128 \rightarrow 768$ (38.6M $\rightarrow$ 6.5M)	
MLM head	tied to the word-embedding table	
normalisation	post-LN	
sequence length	128, contiguous packing (no padding, hence no attention mask)	
masking (train)	BERT 80/10/10 scattered	geometric spans, $p=0.2$, clipped at 10
masking (eval)	scattered	scattered

Table 2: Architectures. The tokenizer is `allenai/OLMo-1B-hf` with an added [MASK] (id 50280, vocab 50281). There is no [CLS]: OLMo has none and NSP was dropped, so downstream classification mean-pools the encoder states.

Parameter split of the v2 model: FFN 16.53M (59%), embedding and projection 6.53M (23%), attention 4.72M (17%), norms 0.02M. Attention holds roughly half the share it does in BERT-base ($\approx 33\%$): with $n_{\text{heads}} \cdot d_k = 256$ against `d_model` = 768, each block writes a rank- ≤ 256 update into a 768-dimensional stream. This asymmetry recurs below.

2.2 Data

Corpora were tokenised to packed `uint16` memmaps with contiguous packing (documents joined by EOS, every block full): WikiText-103 119M tokens, UltraFineWeb-en 341M, TinyStories 475M, Cosmopedia 403M, IMDB 23M.

2.3 Training

bf16, AdamW, micro-batch $16 \times$ gradient accumulation 16 (effective batch 256). Measured VRAM fit on the 3050 Ti, which fixed the micro-batch:

micro-batch	peak VRAM	ms/step	verdict
16	1.84 GB	91	safe default; leaves the machine usable
32	3.25 GB	188	works if the GPU is otherwise idle
48	4.63 GB	507	spills to WSL2 shared memory
64	6.03 GB	1751	heavy spill

2.4 Evaluation protocol

Every cross-model number in this report is scored on **one fixed held-out mixture**: the tail 1% of each Phase-A corpus, token-weighted 55/35/10 (Cosmopedia / UltraFineWeb / TinyStories), with an identical RNG seed so that all models see byte-identical masked inputs. Two masking schemes are reported: **scattered** (BERT 15%, 80/10/10) and **span** (geometric $p=0.2$, clipped at 10, mean span 3.80). Fifteen batches of 16×128 ; eval noise is $\approx \pm 0.05$ nats.

This protocol was introduced late. Adopting it retroactively converted an entirely uncomparable set of run logs into a comparable table, and Section 8 quantifies what its absence had been costing.

3 What worked

3.1 Data curriculum: the largest single lever in the project

The staged curriculum—WikiText-103, then UltraFineWeb, then a replay mixture with TinyStories and IMDB, then a Cosmopedia-weighted mixture—produced the two largest improvements we recorded, both of them corpus changes rather than architecture changes.

checkpoint	data added	scat_loss	Δ
ckpt	WikiText-103, 119M	5.9866	—
ckpt2	+ UltraFineWeb	3.0389	−2.948
ckpt3	+ TinyStories + IMDB (replay)	2.7089	−0.330
ckpt_v2	+ Cosmopedia (55% weight), 734M	2.3280	−0.381
ckpt_v2_anneal_cosmo	4k-step Cosmopedia anneal	2.2738	−0.054

Table 3: Data curriculum on the fixed held-out mixture (§2.4). The architecture change between rows 3 and 4 is analysed separately in §9; the corpus change is the dominant term.

Cosmopedia was weighted highest in Phase A on an explicit argument: it was the only genuinely *new* data, since ckpt3 had already seen UltraFineWeb for ≈ 2.5 epochs, near the point where repeats stop paying. The per-corpus breakdown below confirms this paid—Cosmopedia improved by 0.81 nats while everything else moved by a quarter of that.

3.2 Replay mixing works, and we have the control

Continued pretraining on a new corpus normally causes catastrophic forgetting. We mixed each new phase with replay from earlier corpora. IMDB provides an unintentional but clean negative control: it was dropped entirely from the v2 mixture while the other three corpora were retained at 55/35/10.

checkpoint	UltraFineWeb	TinyStories	IMDB	Cosmopedia
ckpt (WikiText only)	5.9660	5.4163	6.0371	6.3295
ckpt2 (+UFW)	2.8882	2.4936	3.8784	3.4847
ckpt3 (+TinyStories+IMDB, replay)	2.9511	1.0227	2.9255	3.1006
ckpt_v2 (+Cosmopedia, replay)	3.2132	1.2459	3.8733	2.2875
ckpt_v2_anneal_cosmo (final)	3.3605	1.3430	3.9433	2.1129

Table 4: Per-corpus held-out scattered MLM loss. Each column is a separate held-out split; columns are not comparable to each other, only down.

Reading the ckpt3 $\rightarrow$ ckpt_v2 transition against the replay weight:

corpus	weight in the v2 mixture	Δ loss
Cosmopedia	55% (new)	−0.813
UltraFineWeb	35%	+0.262
TinyStories	10%	+0.223
IMDB	0%	+0.948

Any replay at all held forgetting to ≈ 0.25 nats; zero replay cost 0.95 nats, close to four times as much, and dropping from 35% to 10% cost nothing measurable. The practical reading is that the replay *floor* matters much more than the replay *fraction*: a token budget of 10% is enough, and 0% is not.

The same table also characterises what annealing does, honestly. The Cosmopedia anneal improved Cosmopedia by 0.175 nats and degraded all three other corpora (UltraFineWeb +0.147,

TinyStories +0.098, IMDB +0.070). Annealing *specialises*; it does not uniformly improve. This is why the anneal simultaneously measured -0.054 on the Phase-A mixture and $+0.057$ on IMDB, an apparent contradiction at the time.

3.3 Span masking

Following SpanBERT, the v2 line trains on geometric span masking ($p=0.2$, clipped at 10, mean span 3.80) and evaluates on scattered masking so the logged accuracy stays comparable with earlier runs. Span masking removes the ability to infill from immediate neighbours, forcing representation rather than local autocomplete.

It worked, on two independent readings. First, the model improved *more* on the harder task than on the easier one: from `ckpt3` to the final model, scattered accuracy rose 4.6 pp while span accuracy rose 6.7 pp. Second, the intrinsic difficulty gap between the two schemes narrowed, from $+1.932$ nats for the scattered-trained `ckpt3` to $+1.640$ nats for the span-trained final model—a 0.29 nats narrowing that is six times eval noise.

That gap is large enough to be worth stating on its own, because it is a frequent source of false alarms. Measured on one fixed model with identical data and seed:

masking scheme (mlm_prob 0.15)	mean span	mlm_loss	masked_acc	vs scattered
scattered	1.16	2.3655	0.5814	—
uniform 2–4	2.94	3.5002	0.4362	+1.135
geometric $p=0.2$	3.64	4.1054	0.3662	+ 1.740
geometric $p=0.1$	5.36	4.4758	0.3186	+2.110

A model that trains on spans and evaluates on scattered will show train and validation curves ≈ 1.7 nats apart *by design*. Anyone reading those logs without this table would diagnose a catastrophic bug.

3.4 WSD with the decay phase left unspent

`ckpt3` used cosine decay to zero and demonstrated that it bought nothing: the best validation loss arrived at step 6,400 while the learning rate was still at 55% of peak, and the fully annealed endpoint was *worse* than the mid-run best. Every later run therefore used Warmup–Stable–Decay with `decay_frac = 0`, ending flat.

Two benefits followed. The run stays extendable without a re-warm, and the decay phase becomes a separate, controllable experiment—which is exactly what made the corpus-controlled annealing comparison of §7 possible at all. Spending the decay is a decision; it should not be a side effect of the schedule.

3.5 Installing an induction circuit into a masked language model

We coerced the encoder into an autoregressive generator—append three [MASK] tokens, supervise and commit slot 1, slide the window—as a probe of what the MLM had absorbed. Two capabilities were missing and both were installable in minutes of CPU-class fine-tuning.

- **Termination.** The base model never emits EOS in a generation loop. Oversampling EOS positions (30% of supervised slots) during a 400-step continuation fine-tune fixes this.
- **Copy-from-context is absent at baseline: 0.8%.** Induction requires a two-layer circuit—a previous-token head feeding a copy head—and four heads per layer leave no room for one to form incidentally under a masked objective. A 600-step fine-tune on two-entity templates (25% of the batch, the rest replay) raises copy accuracy to **93.3%** on trained-style prompts and **80.0%** on 200 entities held out of the training pool, in 6.5 minutes.

The generalisation to held-out entities is what makes this a circuit rather than memorisation. The result is that the capability was *cheap and absent*: nothing in 2.1B tokens of MLM pretraining gave the model a reason to build it, and once given a reason it built it in 6.5 minutes.

Two honest caveats. The fine-tune over-corrects at high copy weight—“*We went to Paris last year with Maria. I loved chatting with*” completes to “*Paris last year with Maria*”. And there is an open failure: generation loops on prompts shorter than 8 tokens, because training sampled context length uniformly over [8, 125] while generation *starts* at the prompt length. All four sub-8-token prompts break; all five longer prompts are clean. The fix (log-uniform context sampling from length 1) is identified but not implemented.

3.6 The diagnostic tooling

The tools in `tools/` are themselves a result—arguably the most reusable one, and the *pattern* they share is treated as a contribution in its own right in §6.1. Three earned their cost repeatedly: `span_eval.py` (the fixed-protocol evaluation of §2.4), `layer_contrib.py` (causal ablation cost, single and pairwise, §5), and `audit_eval.py` (which mechanically checks the failure modes of §8). Each converted a recurring argument into a number.

4 Function-preserving grafts in post-LN transformers

4.1 The problem

Growing a trained network by adding layers—progressive stacking, bert2BERT, Net2Net—is meant to be near-free: the new layer starts from a useful function and training continues from a good point. The standard trick for making it *exactly* free is to zero the new block’s output projections so that the block is an identity at step 0.

This is folklore for **pre-LN** architectures, where a block computes $x + f(\text{LN}(x))$ and $f \equiv 0$ genuinely is the identity. Our model is **post-LN**:

$$h = \text{LN}_1(\text{MHA}(x) + x), \quad (1)$$

$$\text{out} = \text{LN}_2(\text{FFN}(h) + h). \quad (2)$$

With $W_O = 0$ and $W_{\text{out}} = 0$ this collapses to $\text{out} = \text{LN}_2(\text{LN}_1(x))$ —two extra LayerNorms, not an identity. We measured what that costs and found the folklore is recoverable in post-LN, subject to two conditions that are not usually stated.

4.2 Measurement

All variants below are built from the same trained 4-layer checkpoint (`ckpt3`) by appending *one* block, and are scored under the protocol of §2.4. Because building a v2-package model also swaps ReLU for GELU, Δ is reported against the GELU-only reference so the activation change is factored out.

variant (one block appended to a trained 4L stack)	scat_loss	Δ vs ref	max $ \Delta \logit $
<code>ckpt3</code> , 4L, ReLU — the source model	2.7089	—	—
GELU swap only, still 4L — the reference	2.9776	0.000	—
$W_O=0$, $W_{\text{out}}=0$, LN affines reset to (1,0)	2.9776	+0.000	0.0013
$W_O=0$, $W_{\text{out}}=0$, LN affines <i>copied from donor</i>	7.1016	+4.124	37.7
only $W_{\text{out}}=0$ (attention live), LN copied	7.7290	+4.751	—
full copy-stack — <i>what this project actually did</i>	8.5488	+5.571	—

Table 5: Decomposing the cost of adding a layer to a trained post-LN stack.

The mean logit magnitude is 3.82, so $\max |\Delta \text{logit}| = 1.3 \times 10^{-3}$ is float32 round-off (a relative error of 3.3×10^{-4}): the corrected graft is **exactly** function-preserving. The naive graft destroys the model.

4.3 Why it works, and where it stops working

A zero-output post-LN block computes $\text{LN}(\text{LN}(x))$, which with unit affines equals $\text{LN}(x)$ —a per-token standardisation of x . That is *not* x . But LayerNorm is invariant to per-token affine rescaling,

$$\text{LN}(a x + b \mathbf{1}) = \text{LN}(x), \quad a > 0, b \in \mathbb{R}, \quad (3)$$

and $\text{LN}(x)$ is itself exactly such a rescaling of x (with $a = 1/\sigma$, $b = -\mu/\sigma$). So if the *next* consumer of the stream is another LayerNorm, the standardisation is annihilated and the graft is invisible.

At the **top** of the stack the only consumer is the encoder’s final LayerNorm, and (3) applies exactly. **Mid-stack**, the next block’s attention reads the residual stream *directly*—attention logits scale as the square of the stream magnitude—while the graft has just stripped the previous block’s learned output scale (γ, β) . Equation (3) no longer applies:

insertion point of the zero-graft block (5L/3072)	scat_loss	Δ
nowhere — 4L reference	2.9776	—
position 0 (before all trained blocks)	3.0646	+0.087
position 2 (mid-stack)	3.2021	+0.225
position 4 (appended at top)	2.9776	+0.000

Position 0 is cheaper than position 2 only because the embedding-plus-positional stream it re-standardises is less shaped by a learned LayerNorm than a mid-stack activation is.

4.4 The counter-intuitive term

The dominant error is **copying the donor block’s LayerNorm affines**: +4.12 nats of the +5.57 nats total. Every instinct in progressive stacking says to copy everything from a trained layer—that is the entire argument for stacking over random initialisation. For the LayerNorm affines specifically, copying is worse than resetting by more than four nats, because γ and β encode *what scale the next layer expects*, and a newly inserted block inherits the wrong expectation.

4.5 Practical rules

- (a) **Post-LN**: zero-init grafting is exact *only at the top of the stack*, and only with $W_O = 0$, $W_{\text{out}} = 0$, and **LayerNorm affines reset to** (1, 0). Mid-stack, budget ≈ 0.2 nats.
- (b) **Copy weights, reset norms**. Inherit the residual-branch parameters; do not inherit γ and β .
- (c) **Verify with logits, not loss**. Our first check compared loss on a small batch and read a plausible-looking delta. Comparing $\max |\Delta \text{logit}|$ against the mean logit magnitude turns a judgement call into a pass/fail: 1.3×10^{-3} versus 37.7 is not a debate. The verification belongs *inside* the tool that performs the graft.
- (d) **A destroyed output does not mean destroyed knowledge**. `ckpt_v2_init` scored 8.26 nats. It recovered to 2.71 nats—its parent’s level—within 1,000 steps, and to 2.38 nats by step 14,200, a level `ckpt3` needed $\approx 42,000$ cumulative steps to reach. Loss immediately after a graft is a very poor proxy for how much transferred.

5 Depth at 28M: the sixth layer

We grew 4 layers to 6 by cutting `d_ff` from 3072 to 1792 and spending the savings on depth. Layer 5 pulled its weight. **Layer 4 did not.**

5.1 Ablation cost: a causal metric

Correlational metrics (utilisation, effective rank, head similarity) describe what a layer *looks like*. We wanted what it is *worth*, so we defined **ablation cost**: zero the layer’s output projections (W_O , l_2 , b_2), leaving the residual path intact, and measure the increase in held-out loss.

layer	0	1	2	3	4	5
ablation cost (nats)	1.186	1.683	1.749	1.594	0.073	0.963

Layer 4 is worth 7.6% of what its neighbour layer 5 is worth, and 1.0% of the stack’s total ablation cost, while holding an identical 12.7% share of the parameters.

5.2 Three initialisations, one answer

initialisation of layer 4	steps trained	final ablation cost
stacked copy of a trained layer	9,600	≈ 0
zero-init graft, $5\times$ LR, no weight decay	4,600	≈ 0
standard $\mathcal{N}(0, 0.02)$, <i>rest of the network frozen</i>	18,800	+0.062

The third run is the decisive one. Freezing embeddings, the MLM head and blocks 0, 1, 2, 3, 5, leaving 3.545M of 27.952M parameters trainable (12.7%), removes every escape route: the rest of the network *cannot* route around layer 4, so whatever it can learn, it must. It climbed from 0 to +0.062 nats over 18,800 steps and flattened, which is $\approx 6\%$ of layer 5’s contribution.

Verdict: position, not initialisation. At this depth and scale there is no work available at slot 4 that existing redundancy in layers 1, 2 and 5 does not already absorb. Transplanting the frozen-trained layer back into the joint run improved loss by -0.056 nats instantly—and joint training gave most of it back within 200 steps. The joint optimiser actively *prefers* the configuration in which layer 4 does nothing.

5.3 Pairwise marginals change how ablation should be read

Single-layer ablation cost is not a clean attribution. We measured the full pairwise matrix, with $\text{marginal}(i \mid j) = \text{cost}(\{i, j\}) - \text{cost}(\{j\})$:

$j =$	0	1	2	3	5
marginal(4 $\mid$ j)	-0.012	+0.031	-0.019	-1.891	+0.005

Layer 4 is worth ≈ 0 given *any* other layer removed, so redundancy is not the explanation—nothing was covering for it. But at $j = 3$, **removing the useless layer 4 halves the apparent cost of removing layer 3** ($3.437 \rightarrow 1.545$).

The mechanism is fragility amplification. Ablating layer 3 produces an off-distribution residual stream; layer 4, trained only on in-distribution input, amplifies that corruption rather than absorbing it. Layer 3’s headline cost of 3.44 nats is therefore $\approx 55\%$ damage routed through layer 4 and only $\approx 45\%$ its own contribution.

Rule: single-layer ablation cost over-attributes to layers sitting below a fragile downstream layer. For pruning decisions the pairwise marginal is the number wanted, and it can differ from the single by a factor of two.

6 Diagnostics: zero-cost telemetry, and what it can and cannot tell you

6.1 Zero-cost live diagnostics

On a 4 GB laptop the GPU *is* the training run. Any diagnostic that wants the GPU competes with the thing it is diagnosing, and the honest measure of model quality—a downstream fine-tune—costs 8.3 minutes of exclusive GPU, with the full benchmark suite at 13.2–16.6 minutes. At that price it is affordable perhaps twice per run, which means flying blind for hours and learning everything in the post-mortem.

The pattern we settled on extracts continuous internal-health telemetry from a live run at approximately zero marginal cost. It has three parts, and it is the most transferable piece of engineering in this project.

1. Atomic checkpoints make the run readable from outside. `atomic_save` writes through `os.replace`, which is atomic on POSIX. An external process can therefore read `last.pt` at any instant during training and is guaranteed a complete, self-consistent checkpoint—never a torn write. This is the load-bearing piece: diagnostics need not live *inside* the training process, so they cannot slow it down, crash it, or be lost with it.

2. Score on the idle device. `span_eval.py`, `attn_health.py` and `ffn_slack.py` default to CPU precisely so they never contend for VRAM. Measured wall-clock on the final checkpoint:

diagnostic	device	cost	what it reports
<code>ffn_slack.py</code>	CPU	8.9 s	slack, never-positive fraction, utilisation
<code>attn_health.py</code>	CPU	14.8 s	per-head entropy, <code>out_rank</code> , head similarity
<code>layer_contrib.py</code>	CPU	22.2 s	causal ablation cost per layer
<i>SST-2 fine-tune</i>	<i>GPU, exclusive</i>	<i>8.3 min</i>	<i>the actual downstream number</i>
<i>full benchmark suite</i>	<i>GPU, exclusive</i>	<i>13.2–16.6 min</i>	<i>SST-2 + STS-B, probe + fine-tune</i>

Table 6: Diagnostic cost against evaluation cost. The correlational diagnostics are $\approx 60\times$ cheaper *and* run on a different device, so their marginal cost to the training run is essentially zero.

We ran these on a 20-minute loop for the entire project, accumulating ≈ 100 readings per metric across 35 GPU-hours. That cadence is what made trends legible. Individual readings are noisy enough that layer 4’s trajectory was mis-called repeatedly from single cycles—“converged”, then “accelerating”, then “regression”, all wrong. The fix was fitted slopes over multi-cycle windows, and those windows exist only because each reading cost nine seconds of idle CPU rather than a quarter-hour of the GPU.

3. Stream the already-free signals into the training log itself. `layer_probe` runs one extra forward pass on the eval batch that is already resident, materialising attention probabilities only for the named layers, and writes `l4_ent`, `l4_np`, `l5_ent`, `l5_np` into `metrics.jsonl` next to the loss. The dashboard (`tools/serve_dashboard.py`, standard library only, no dependencies) plots them live. Per-layer health stops being a post-mortem and becomes a curve one watches.

This is what made the layer-4 graft legible while it was happening. Attention entropy near $\log T = 4.85$ means a head is averaging rather than selecting; `never_pos` near 1 means the FFN is switched off. Watching `l4_ent` sit at 3.9 said within minutes—not hours—that the fresh layer was not differentiating.

The transferable rule. Make the run readable from outside (atomic writes), score on whichever device is idle, and stream into the log only those signals that are already nearly free.

On constrained hardware this turns “I will find out in the post-mortem” into a live feed, for approximately nothing.

One caveat, which the rest of this section is about: everything above concerns **delivery**, not **validity**. These metrics earned their keep—effective rank designed the v2 architecture and dead-unit counting corrected how it was pruned (§6.2)—but each needed calibrating first, and three of them gave confidently wrong readings until their nulls and degenerate cases were audited.

6.2 What the cheap metrics got right

Before the failures, the record: these tools were mostly right, and they were load-bearing. The entire v2 architecture was designed from correlational readings, and the readings held up.

Effective rank designed the model. `ffn_rank.py` measured the spectrum of FFN activations and of the residual stream on `ckpt3`, and the two readings pointed in opposite directions—which is exactly what made them actionable:

	reading	conclusion
FFN activations, $r@99\%$	133 / 834 / 744 / 866 of 3,072	width is $3.5\times$ oversized
residual stream, eff. rank	149 $\rightarrow$ 269 $\rightarrow$ 483 $\rightarrow$ 523 of 768, still climbing	depth has headroom

Width was cut $3072 \rightarrow 1792$ and the savings spent on depth. The check afterwards closed the loop: re-measuring the trained v2 gave $r_{99}/d_{ff} \leq 0.56$ on every layer—width was *still* not the binding constraint even at 1792, so the cut was safe rather than merely survivable. Predicted, acted, verified. That is the strongest endorsement a correlational metric can get, and this one earned it.

Dead-unit counting changed how the prune was scored. `dead_neuron_check.py` found 2,697 of 3,072 layer-0 units that never fired once over 2.0M tokens under ReLU. That motivated the GELU swap—but the more valuable consequence was procedural: pruning importances were then scored *under GELU rather than ReLU*, which kept 1,501 of layer 0’s 2,755 ReLU-dead units (96.2% of the retained importance mass). Scoring under the wrong activation would have discarded exactly the capacity the activation swap existed to recover.

The same tool taught the sample-size lesson: at 8,192 tokens layer 0 read 2,795 dead; at 2.0M tokens it read 2,697, with explicit $<1/100k$ and $<1/10k$ columns separating genuinely dead units from rare-token detectors. Pruning the latter removes memory, not redundancy.

Ablation cost settled a question three other metrics could not. `layer_contrib.py` is the causal one, and it worked: slack, effective rank and head similarity had all pointed at layer 4 ambiguously, and ablation gave a number— $+0.073$ nats against layer 5’s 0.963—that survived three initialisations and 40+ readings (§5).

Head similarity worked once centred. After the fix, layer 3 still read 0.95 and *was* genuinely a duplicated pair, while layers 4 and 5 fell to 0.13 and 0.33—correctly reclassified from “duplicated” to “untrained”. The metric was recovered, not discarded.

FFN slack learned to distinguish its own two causes. It now annotates layers ! (bi-modal dead-zone mass—units parked below the activation threshold) versus * (genuinely dense, a real width-pressure candidate). The same number, two opposite meanings; the tool reports which.

So the honest summary is not that these metrics lie. It is that each needed **a null and a degenerate-case audit** before its readings meant anything—and that even fully calibrated, they answer a different question than ablation does. The next three subsections are what that cost when we skipped it.

6.3 An unmeasured null: head similarity read ≈ 0.97 at random init

The maximum pairwise cosine between heads’ attention matrices read 0.98 on layer 3, apparently indicating duplicate heads. It does not: **an untrained model reads 0.95–0.99 as well**. Attention rows are probability distributions over 128 positions; an undifferentiated head sits near-uniform, and the shared $1/T$ floor dominates the cosine. The metric could not distinguish “duplicated heads” from “untrained heads”—precisely the two cases it existed to separate.

Centring across heads before taking the cosine restores a meaningful null. Measured on a freshly initialised model: **raw 0.95–0.99, centred -0.23 to -0.07** . The null is *negative* because four centred vectors sum to zero, so undifferentiated heads are mutually anti-correlated.

A second bug compounded the first: zeroing the diagonal before the argmax both won the argmax and clamped the reported maximum, so the tool reported the nonsense pair $h_0 \sim h_0$ at 0.00. The diagonal must be $-\infty$.

After the fix, layer 3’s 0.98 survived (genuinely duplicated) while layers 4 and 5’s 0.86 and 0.98 collapsed to 0.18 and 0.27—they were untrained, not duplicated. Every conclusion drawn from the uncentred metric had to be discarded.

6.4 The same failure one level up: latent-target pretraining

We ran a data2vec/JEPA-style latent-prediction branch (EMA teacher, stop-gradient, narrow predictor, momentum $0.996 \rightarrow 0.999$) for 15,258 steps on 500M tokens. Every internal metric was encouraging.

step	centred cos	raw cos	constant-predictor baseline	eff. rank	target std
200	0.0357	0.886	0.916	240.9	0.349
8,600	0.6161	0.944	0.907	232.9	0.363
15,200	0.6256	0.940	0.895	234.8	0.382

Note the raw cosine at step 200: **0.886, against a constant-predictor baseline of 0.916**. A predictor that ignored its input entirely and emitted the dataset mean would have scored *higher*. Representations are anisotropic; almost any two vectors are similar. The honest number was 0.0357.

We caught that one. What we did not catch until downstream evaluation is that the *whole class* of internal metric was uninformative here: centred cosine rose $17\times$, effective rank held steady, target standard deviation improved monotonically—and the downstream result was indistinguishable from the far simpler MLM line (§11).

6.5 A degenerate case: zero-output layers poison run-level aggregates

FFN-slack and attention-health tools reported `slack = 100%`, `util = 1.00`, `out_rank = nan` for a freshly grafted layer whose output projection is identically zero. Those rows entered the run-level means and flipped an automated verdict to “OK” for several monitoring cycles. Any health metric of the form “*how much of this layer’s output matters*” is undefined when the output is zero by construction; the tool must detect and exclude that case rather than average it in.

6.6 The general lesson

Every proxy metric needs its null measured on an untrained model of the same architecture, and its degenerate cases enumerated, before it is trusted once. That is a calibration cost, not a verdict: once paid, effective rank designed a working architecture, dead-unit counting corrected the pruning procedure, and centred head similarity separated duplicated heads from untrained ones. Every one of these was cheap, and every one earned its place.

What survives as a genuine limitation is narrower and more interesting. Ablation cost is the only diagnostic we never had to retract, because it is defined by an *intervention* on the model rather than by a statistic of its activations—and it is $\approx 40\times$ more expensive per reading. Everything cheap is a proxy for “is this layer doing something”, and a proxy can be perfectly calibrated and still answer a different question than “what happens if I remove it”. §6.4 is the extreme case: metrics that were correctly read, monotonically improving, and predicted nothing about the outcome we cared about.

Use the cheap metrics continuously to see *what the model is doing*; use ablation sparingly to find out *what it is worth*. They are not competing instruments.

7 Annealing: corpus dominates schedule

Phase A used WSD with `decay_frac = 0`, deliberately leaving the decay phase unspent. We then spent it twice from the *identical* checkpoint with *identical* learning rate, schedule, seed and step count (cosine $2 \times 10^{-4} \rightarrow 0$, no warmup, 4,000 steps), varying only the corpus.

anneal corpus	own val_loss	held-out result vs parent	outcome
TinyStories	0.9989	fell <i>below ckpt3</i> within 1,800 steps	abandoned
Cosmopedia	2.0487	-0.054 nats, $+1.18$ pp span acc	kept; final model

TinyStories produced by far the lowest validation loss the project ever recorded, and it was pure artefact: TinyStories is deliberately constructed with a $\approx 1,500$ -word vocabulary at a three-to-four-year-old reading level. Annealing on it *narrows the lexicon*; it does not teach fluency. It lost three quarters of v2’s entire advantage over `ckpt3` in 800 steps. Cosmopedia—the same schedule on synthetic textbook prose—did **$11\times$ less damage** and net helped.

Because everything except the corpus was held fixed, this isolates data register as the causal variable. **Annealing is a data-selection decision, not a schedule trick, and the loss on the anneal corpus itself carries no information about whether it worked.**

8 Evaluation validity

Late in the project a reported validation loss of ≈ 2.04 looked implausibly good—a well-trained 124M-parameter GPT-2 on 10B tokens lands near 3.3, and we would have been beating it with a quarter of the parameters and 7% of the data. We audited it against the standard failure modes.

1. It was a different task. Masked language modelling is not autoregressive language modelling; bidirectional context makes the token far easier to predict. There was no valid comparison to GPT-2 in the first place. A category error rather than a bug, but the most consequential item on the list.

2. The eval mixture, not the model, was doing most of the work. For the *same* checkpoint:

evaluation mixture	scat_loss
Cosmopedia only (what that run logged)	2.0487
Phase-A mixture, 55/35/10 (the protocol)	2.2738

0.225 nats from the evaluation set alone—over half the size of the entire project’s pretraining improvement (0.435 nats). Every run in this repository evaluates on its own training mixture by default, which makes the `val_loss` column of any cross-run table meaningless.

3. Cross-run contamination is real and easy to miss. The split is contiguous (validation is the tail 1%), so it is clean *within* a run. But `limit_tokens` changes where the boundary falls, and weights are inherited across runs:

```
tinystories  n = 73.4M  train=[0, 72.6M)  val=[72.6M, 73.4M)
prior run           train=[0, 165.3M) → VAL INSIDE PRIOR TRAIN: CONTAMINATED
```

v2’s TinyStories validation range sits entirely inside `ckpt3`’s training range, and v2 inherited `ckpt3`’s weights. UltraFineWeb was clean; TinyStories was not.

4. Averaging bias was real but negligible. Mean-of-batch-means 2.2743 against token-weighted 2.2760: a -0.0016 nat bias. Worth checking once; not worth worrying about.

5. Data was not pathologically repetitive. 0 of 320 validation blocks byte-matched any of 199,989 sampled training blocks. The top-8 scored tokens account for 22.5% of scored positions across 2,357 distinct types—normal for English.

6. The generalisation gap is small but nonzero. Train-range 2.1427 against held-out 2.3251: $+0.18$ nats and -1.2 accuracy points. Consistent with a capacity-bound, zero-dropout model that is not overfitting.

9 Width–depth reallocation

Three measurements on `ckpt3` motivated the v2 architecture:

- **`d_ff` was $\approx 3.5\times$ oversized.** 99% of FFN activation variance fit in 133–866 of 3,072 directions, depending on layer.
- **Layer 0 had 2,697 of 3,072 units that never fired once** over 2.0M tokens under ReLU.
- **Residual-stream effective rank was still climbing at the last layer** ($149 \rightarrow 269 \rightarrow 483 \rightarrow 523$ of 768), so depth had headroom where width did not.

We therefore cut `d_ff` to 1792, swapped ReLU for GELU—and, importantly, scored the pruning importances *under GELU*, because scoring under ReLU would discard units whose pre-activations are negative (2,755 of them in layer 0) that GELU passes with a small non-zero output—and spent the savings on two extra layers, landing at 27.952M under the cap.

The width cut cost almost nothing, as predicted: structured pruning $3072 \rightarrow 1792$ cost $+0.083$ nats on top of the activation swap. After training, $r_{99}/d_ff \leq 0.56$ on every layer—width was still not the binding constraint even at 1792.

The depth it funded returned almost nothing: layer 5 pulled its weight, layer 4 did not (§5), and the end-to-end gain did not survive to the downstream task (§11).

This is what scaling laws predict—performance depends on parameters, data and compute, and only weakly on architectural shape. We ran the experiment carefully enough to confirm rather than assume it, but the honest summary is that **reallocating a fixed parameter budget between width and depth is a rounding error compared with changing the budget**.

10 Engineering results

Freezing 87% of parameters buys 1.6×, not 8×. The frozen-stack experiment trained only layer 4—12.7% of parameters—at 0.858s/step against 1.397s/step for full joint training: 1.63×. The reason is FLOP structure, not parameter count. The forward pass is unchanged (all six layers), and the backward pass must still traverse layer 5 to reach layer 4; only the four layers *below* the trainable one skip backward. Predicted 1.8×, measured 1.63×. **Count the backward passes you actually skip, not the parameters you froze.**

Budget by steps, not wall-clock, on a machine that sleeps. A `-max-seconds 10800` budget using `time.time()` counted sleep and terminated a run at step 1,851 of 8,800 after 37 minutes of real compute.

A watchdog inside the thing it watches is not a watchdog. Every recovery mechanism we built initially lived inside the session that sleep suspended, dying at the same instant as the training it existed to observe. Moving it to cron covered crashes but not Windows sleep, which *destroys* the WSL2 VM—confirmed by two distinct boot IDs with an 8h38m hole in `journalctl -list-boots`. There is no kernel running to fire cron.

A watchdog that spawns a daemon can disable itself. The supervisor held its lock on file descriptor 9; `setsid` children inherit open descriptors, so the long-lived dashboard *it* spawned captured fd 9 and held the lock for its lifetime. Every subsequent cron tick died at `flock -n 9` and exited 0, silently, for ≈ 5 hours. The one recovery test that passed did so only because the dashboard alive at that moment had been started from a shell. **The act of the supervisor launching a daemon is what disabled the supervisor—that test could not have caught the bug however carefully it was run.** Verifying a watchdog once tells you nothing about whether it stays armed.

`${VAR:-default}` substitutes on empty, not merely on unset. `BOOST_LAYERS=""` fell through to the default and silently launched two runs with a 5× learning rate that had been explicitly ruled out. `${VAR:-default}` is the correct form. Cost: two killed runs and two checkpoint restores.

Grep for a process name and you will match your own shell. A GPU-wait loop using `ps | awk '/pretrain_v2/'` matched any command that merely *mentioned* the string, including the monitoring commands, and wedged forever. Poll the resource (`nvidia-smi` VRAM), not the name.

Sparse-file apparent size is not disk usage. Packed `.bin` memmaps are allocated at target size and written incrementally; the holes cost zero real blocks. Truncating them to `tokens_written` reduced apparent size from 3.7 GB to 2.4 GB and freed nothing—and requires updating the manifest in lockstep, since the dataset memmaps with `shape=(target_tokens,)`.

11 Scoreboard, and the central negative result

11.1 Pretraining progressed clearly

checkpoint	step / arch	scat_loss	scat_acc	span_loss	span_acc
ckpt — WikiText-103	8,800 4L/3072	5.9866	0.2531	6.9932	0.1695
ckpt2 — +UltraFineWeb	20,789 4L/3072	3.0389	0.4819	4.9318	0.2884
ckpt3 — +TinyStories+IMDB	12,964 4L/3072	2.7089	0.5295	4.6408	0.3118
ckpt_v2_init — graft, untrained	0 6L/1792	8.2635	0.0125	8.4851	0.0075
ckpt_v2 @ 14.2k	14,200 6L/1792	2.3828	0.5661	4.0408	0.3588
ckpt_v2 @ 18.4k (pre-anneal)	18,400 6L/1792	2.3280	0.5759	3.9809	0.3671
ckpt_v2_14only — frozen-stack L4	18,800 6L/1792	2.3136	0.5781	3.9658	0.3695
ckpt_v2_anneal_cosmo — final	+4,000 6L/1792	2.2738	0.5759	3.9135	0.3789

Table 7: All rows scored under the fixed protocol of §2.4. **ckpt3** to final is -0.435 nats scattered, -0.727 nats span, $+6.7$ pp span accuracy— $\approx 9\times$ eval noise and unambiguously real. The latent-target branch is absent because it never trained an MLM head; scoring it under MLM reads 11.98 nats and means nothing.

11.2 Downstream did not

SST-2 dev is 872 examples: SE ≈ 1.2 pp. STS-B dev is 1500: SE ≈ 1.7 pp.

model	SST-2 standalone	SST-2 probe	SST-2 ft	STS-B probe	STS-B ft	STS-B 0-shot
ckpt3	0.8567	0.7741	0.8509	0.5971	0.4199	0.5610
ckpt_jepa	0.8429	0.7362	0.8647	0.6035	0.6111	0.4600
ckpt_v2_anneal_cosmo	0.8601	0.7431	0.8509	0.5853	0.1434 [†]	0.5558
BERT-base (110M)	≈ 0.927					

This is the project’s central negative result. A -0.435 nats improvement in held-out MLM loss produced a **0.0 pp** change on the SST-2 fine-tune harness and $+0.34$ pp (0.3 SE) on the standalone harness. Nothing here is distinguishable from noise: the full spread across three very different pretraining regimes is 0.8509–0.8647, i.e. **1.1 SE**.

Worse for interpretation, **the two SST-2 harnesses disagree by more than any of our interventions moved the number.** The same final checkpoint reads 0.8601 standalone and 0.8509 in the benchmark—a 0.92 pp gap arising from selection protocol and split handling alone, against a 0.34 pp effect size. And the latent-target branch, which the standalone harness ranked last, is the *best* model on both benchmark fine-tunes.

The honest reading is that at 28M parameters on an 872-example dev set, **we built a measurement instrument too coarse to detect what we were optimising.** Resolving a 1 pp difference at this standard error would need roughly four times more labelled evaluation data. That should have been checked before the third architecture experiment, not after the last one.

The remaining ≈ 7 pp gap to BERT-base is capacity, not allocation. The final model’s lineage processed ≈ 2.18 B tokens—78 tokens per parameter, far past any compute-optimal ratio—while validation loss plateaued at the eval-noise floor and zero dropout produced no overfitting. All three signals point the same way.

[†] **stsb finetune = 0.1434** is broken, not a result: a fine-tune cannot legitimately land four times below its own frozen probe (0.5853). It read 0.29–0.32 throughout training, i.e. it diverged. The identical code gave 0.4199 and 0.6111 on other backbones. Undiagnosed.

12 What we would do differently

1. **Size the evaluation before sizing the experiment.** $SE \approx 1.2$ pp on SST-2 dev makes any intervention worth less than ≈ 2.5 pp undetectable. We ran three architecture experiments against an instrument that could not resolve their effects. Either pick a task with a larger dev set, average over seeds, or accept up front that the experiment is qualitative.
2. **Measure every proxy metric's null on an untrained model before trusting it once.** The head-similarity metric read 0.97 at random initialisation. Ten minutes of work would have caught it; instead it invalidated a week of readings.
3. **Build the zero-cost telemetry loop first (§6.1).** Atomic checkpoints plus CPU-side scoring cost a day and made every later experiment observable while it ran.
4. **Prefer causal metrics.** Ablation cost was $\approx 40\times$ more expensive per reading and the only diagnostic never retracted. Budget for it.
5. **Use pairwise marginals, not singles, for pruning decisions.** $\text{marginal}(4 \mid 3) = -1.891$ means single-layer costs can be twice wrong, in the direction that matters.
6. **Fix the evaluation mixture globally on day one.** Per-run `val_loss` on a per-run mixture is not a metric; it is a diary entry. A single frozen held-out set turned an uncomparable results table into a comparable one, retroactively, in one afternoon.
7. **Keep a replay floor, not a replay fraction.** 10% replay retained as well as 35%; 0% cost four times as much.
8. **Do not spend a parameter budget reallocating shape.** At a fixed cap, width–depth trades are noise. If the cap is the constraint, the honest experiment is to raise it.
9. **Verify function-preserving transforms with logits, automatically, inside the tool that performs them.** Our graft tool eventually refused to write when $|\Delta\text{loss}|$ exceeded a threshold. It should have compared logits from the start, and it should have existed before the first graft rather than after the third.

13 Limitations

Single seed throughout: no result here carries an error bar from repetition, only from evaluation-set size. MLM evaluation uses 15 batches (≈ 30 k tokens, ± 0.05 nats).

The §4 graft results are exact at the logit level and do not depend on seed. The §5 layer-4 result replicates across three initialisations and 40+ ablation readings and is the strongest evidence in the report. The §7 $11\times$ ratio is a single pair of runs and should be read as an order of magnitude, not a coefficient. The replay-retention control in §3 is one transition with four corpora at four weights, and the 0% condition was not deliberately designed as a control. All downstream numbers are one fine-tune run each, and §11 argues they should not be over-read.

The comparison to BERT-base is from published figures and is not a controlled comparison: different data, tokenizer, training budget and evaluation harness.

A Reproducing the headline measurements

```
# Section 4 -- the graft ladder, end to end: builds every variant, scores it,
# and runs the logit-level function-preservation check
python tools/graft_ladder.py --device cuda

# fixed cross-model MLM evaluation (the only comparable pretraining numbers)
python tools/span_eval.py \
    --ckpt checkpoints/ckpt_v2_anneal_cosmo/last.pt \
    --baseline checkpoints/ckpt3/last.pt --device cuda
```

```

# causal layer value: single + pairwise ablation cost
python tools/layer_contrib.py --full \
    --ckpt checkpoints/ckpt_v2_anneal_cosmo/last.pt

# graft variants: --new-layer-init {stack,random,zero}, --n-layers, --d-ff
python tools/build_v2_from_ckpt3.py --n-layers 5 --d-ff 3072 \
    --new-layer-init zero --out /tmp/graft.pt

# is the reported val loss measuring what it claims?
# (--data-name is one unbroken line; wrapped here for width)
python tools/audit_eval.py --device cuda \
    --ckpt checkpoints/ckpt_v2_anneal_cosmo/last.pt \
    --data-name "cosmopedia:403451249:403451249,
                ultrafineweb_en:256832885:256832885,
                tinystories:73377468:73377468" \
    --prior-runs "ultrafineweb_en:167000000,tinystories:167000000"

# masking-scheme difficulty offsets
python tools/masking_difficulty.py --ckpt checkpoints/ckpt3/last.pt

# attention health -- head_sim is CENTRED; the raw null is ~0.97
python tools/attn_health.py \
    --ckpt checkpoints/ckpt_v2_anneal_cosmo/last.pt

```

Numeric outputs backing the tables are committed in `results/paper_measurements.json` and `results/graft_ladder.json`.

References

- [1] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL, 2019.
- [2] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, R. Soricut. *ALBERT: A Lite BERT for Self-supervised Learning of Language Representations*. ICLR, 2020.
- [3] M. Joshi, D. Chen, Y. Liu, D. S. Weld, L. Zettlemoyer, O. Levy. *SpanBERT: Improving Pre-training by Representing and Predicting Spans*. TACL, 2020.
- [4] T. Chen, I. Goodfellow, J. Shlens. *Net2Net: Accelerating Learning via Knowledge Transfer*. ICLR, 2016.
- [5] L. Gong, D. He, Z. Li, T. Qin, L. Wang, T.-Y. Liu. *Efficient Training of BERT by Progressively Stacking*. ICML, 2019.
- [6] C. Chen et al. *bert2BERT: Towards Reusable Pretrained Language Models*. ACL, 2022.
- [7] A. Baevski, W.-N. Hsu, Q. Xu, A. Babu, J. Gu, M. Auli. *data2vec: A General Framework for Self-supervised Learning in Speech, Vision and Language*. ICML, 2022.
- [8] J. Kaplan et al. *Scaling Laws for Neural Language Models*. arXiv:2001.08361, 2020.
- [9] J. Hoffmann et al. *Training Compute-Optimal Large Language Models*. arXiv:2203.15556, 2022.
- [10] C. Olsson et al. *In-context Learning and Induction Heads*. Transformer Circuits Thread, 2022.
- [11] N. Reimers, I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP, 2019.
- [12] R. Eldan, Y. Li. *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* arXiv:2305.07759, 2023.

- [13] L. Ben Allal, A. Lozhkov, G. Penedo, T. Wolf, L. von Werra. *Cosmopedia*. Hugging Face, 2024.
- [14] S. Hu et al. *MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies*. arXiv:2404.06395, 2024. (Warmup–Stable–Decay schedule.)